Actualizer_Engine_FDSA_QCA — Usage Manifesto

Pipeline Name: `Actualizer_Engine_FDSA_QCA`
Version: 1.0.0
Author: Mohamed Gamal Eldin Abdelaziz Noureldin
ORCID: 0009-0006-3991-1153
Framework: Consciousness and Prime Base Intelligence (CKT V3_U1)
DOI: 10.5281/zenodo.21420098
Date: July 2026
Status: Production-Ready

> **Abstract.** This manifesto specifies the canonical usage protocol for the `Actualizer_Engine_FDSA_QCA` pipeline — a three-engine composition that integrates the **FDSA Pre-Inference Pruner**, the **QCA Parallel Engine**, and the **Actualizer Engine** into a single, coherent inference-time post-processing layer for large language model decoding. The pipeline operates downstream of any transformer attention mechanism, accepting raw logit vectors and returning actualized, drift-stable tokens. This document is the authoritative reference for pipeline architecture, execution modes, configuration parameters, integration protocols, and benchmark interpretation.

Table of Contents

---

1. Pipeline Philosophy

The `Actualizer_Engine_FDSA_QCA` pipeline exists to solve three entangled failure modes in standard autoregressive LLM decoding:

Failure Mode	Cause	Pipeline Response
Vocabulary Bloat	Softmax distributes mass over thousands of semantically irrelevant tokens	FDSA Pruner eliminates >99% of vocabulary before sampling
Distributional Drift	Token-by-token greedy selection diverges from semantic attractor	Actualizer Engine contracts distribution toward fixed point via Banach iteration
Computational Waste	O(N^2) steering over full vocabulary or full problem space	QCA Parallel Engine partitions into K clusters: O(N^2) to O(N^2/K)

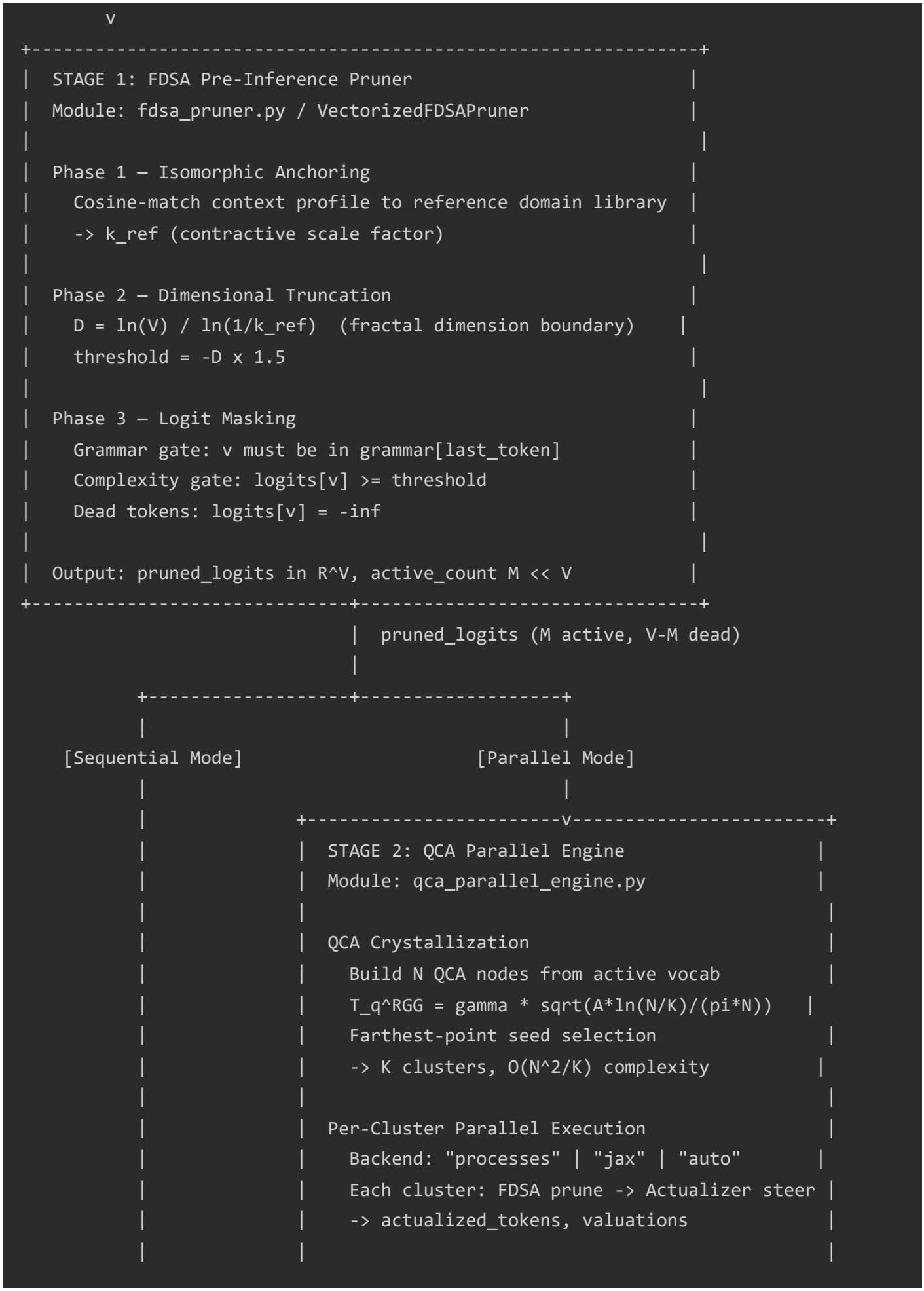
The pipeline enforces the **Five Conceptual Primes** (Order, Justice, Mercy, Knowledge, Power) through the structural entropy function H(R) = Var(alpha) + (sum alpha_i^2 - 1)^2 (V3_U1 §3.3) and the probability-weighted drift trace Tr(D_mu_nu) bifurcation criterion (Theorem 3.3).

> [!IMPORTANT] **Canonical Ordering Law:** The FDSA Pruner MUST execute before the Actualizer Engine. The reversed ordering (Actualizer on full vocabulary, then FDSA masking) wastes approximately 99.85% of Actualizer computation on tokens that will be eliminated, and produces incorrect fixed points (Theorem 2.1, `Unified_Framework.md §2`). This is not configurable — it is a mathematical necessity.

---

2. Architecture Overview

```
[Transformer Attention Engine]
        |
        |  raw logits z in R^V  (V = full vocabulary size)
```


v
STAGE 1: FDSA Pre-Inference Pruner
Module: fdsa_pruner.py / VectorizedFDSAPruner
Phase 1 — Isomorphic Anchoring
Cosine-match context profile to reference domain library
-> k_ref (contractive scale factor)
Phase 2 — Dimensional Truncation
D = ln(V) / ln(1/k_ref) (fractal dimension boundary)
threshold = -D x 1.5
Phase 3 — Logit Masking
Grammar gate: v must be in grammar[last_token]
Complexity gate: logits[v] >= threshold
Dead tokens: logits[v] = -inf
Output: pruned_logits in R^V, active_count M << V
pruned_logits (M active, V-M dead)
[Sequential Mode]
[Parallel Mode]
STAGE 2: QCA Parallel Engine
Module: qca_parallel_engine.py
QCA Crystallization
Build N QCA nodes from active vocab
T_q^RGG = gamma * sqrt(A*ln(N/K)/(pi*N))
Farthest-point seed selection
-> K clusters, O(N^2/K) complexity
Per-Cluster Parallel Execution
Backend: "processes" | "jax" | "auto"
Each cluster: FDSA prune -> Actualizer steer
-> actualized_tokens, valuations

---

3. The Canonical Ordering: FDSA Before Actualization

This is the single most critical constraint of this pipeline.

Why FDSA Must Come First

Let V = vocabulary size (e.g. 32,768), M = active vocabulary after FDSA pruning (typically 50-500).

Correct order (FDSA first, then Actualizer):

N_iters_correct = ceil(log(1/eps) / log(1 + 1/k)) approx k * log(1/eps) over M tokens

Reversed order (Actualizer on full V, then FDSA masking):

N_wasted approx N_max * (V - M) / V approx N_max (nearly all iterations wasted on dead tokens)

For V = 32,768, M = 50, N_max = 32: **99.85% of computation is wasted** in the reversed ordering.

Practical Consequence

```
# CORRECT — used by this pipeline
pruned_logits = fdsa_pruner.prune_numpy(raw_logits, ...)
token = actualizer.steer(pruned_logits, ...)

# WRONG — produces incorrect results, wastes computation
token = actualizer.steer(raw_logits, ...)       # full V dimensions
pruned = fdsa_pruner.prune_numpy(raw_logits, ...)  # masking after the fact
```

> [!WARNING] Do not modify the stage ordering in `pipeline.py`. The ordering is enforced by the `ActualizerFDSAQCAPipeline.run()` method and matches the canonical specification in `Unified_Framework.md §2`.

---

4. Execution Modes: Sequential and Parallel

4.1 Sequential Mode

When to use: Single-step inference, debugging, deterministic benchmarks, latency-sensitive single-request serving.

Flow:

```
Attention Output -> [FDSA Prune] -> [Actualizer Steer] -> Token S*
```

Complexity: O(V) + O(M * N_iters)

Configuration:

```
config = PipelineConfig(execution_mode="sequential", vocab_size=V)
```

Key properties:

- Fully deterministic (given same seed and logits)
- Lowest per-step overhead
- No multiprocessing spawn cost
- Best for latency-critical single-request use cases

4.2 Parallel Mode (QCA)

When to use: Large batch inference, throughput optimization, vocabularies > 5,000, K >= 4 clusters available.

Flow:

```
Attention Output
       |
  [FDSA Prune] -> pruned_logits
       |
  [Build N QCANodes from active vocab]
       |
  [QCA Crystallize] -> K clusters
       |
  [K x (FDSA + Actualizer)] -> actualized_tokens, valuations (in parallel)
       |
  [Synthesis: meta_logits = sum of valuation-weighted votes]
       |
  [FDSA Prune meta_logits]
       |
  [Final Actualizer Snap] -> Token S*
```

Complexity reduction: O(N^2/K) for QCA clustering + K x O((N/K) * N_iters) for parallel steering

Configuration:

```
config = PipelineConfig(
    execution_mode="parallel",
```

```
    K=8,               # number of clusters
    backend="auto",    # "processes" | "jax" | "auto"
    vocab_size=V,
)
```

Backend selection:

Backend	When to use	Notes
`"processes"`	CPU-only, no JAX available	Spawns K worker processes (Windows-safe spawn context)
`"jax"`	JAX installed, GPU/TPU available	Vectorized per-cluster operations on device
`"auto"`	Default	Uses JAX if installed, else processes

> [!TIP] For Windows deployments, the `"processes"` backend uses `multiprocessing.get_context("spawn")` which is safe for use alongside JAX (avoids fork-based JAX corruption). This is already configured in `qca_parallel_engine.py`.

---

5. Configuration Reference

`PipelineConfig` Parameters

Parameter	Type	Default	Description
`vocab_size`	`int`	`1000`	Full vocabulary size V
`mercy_k`	`float`	`0.45`	Contractive scale k in (0,1). This IS the Mercy Prime (V3_U1 §3.3.1-C)
`Q_c`	`float`	`1e-5`	L2 convergence tolerance for Banach iteration
`tau`	`float`	`1.0`	Vacuum Brake temperature tau
`tau_bifurcation`	`float`	`5.0`	Tr(D_mu_nu) threshold for Theorem 3.3 bifurcation gate

Parameter	Type	Default	Description
`max_iters`	`int`	`25`	Maximum contractive mapping iterations per node
`context_type`	`str`	`"logical_coding"`	FDSA context profile for isomorphic anchoring
`execution_mode`	`str`	`"sequential"`	`"sequential"` or `"parallel"`
`K`	`int`	`5`	Number of QCA clusters (parallel mode only)
`backend`	`str`	`"auto"`	Parallel backend: `"processes"`, `"jax"`, `"auto"`
`n_workers`	`int or None`	`None`	Worker processes for backend=processes
`seed`	`int or None`	`42`	RNG seed for reproducibility
`verbose`	`bool`	`False`	Print step-by-step audit log
`prime_weights`	`dict or None`	`None`	Override drift weights {Order, Justice, Knowledge, Mercy}

`context_type` Options and FDSA Domain Anchoring

Context Type	Matched Domain	k_ref	Use Case
`"logical_coding"`	Resistor_Equilibrium	0.35	Code generation, logic tasks
`"mathematical"`	Fermat_Least_Time	0.45	Math proofs, calculation
`"factual_qa"`	Resistor_Equilibrium	0.35	Factual question answering
`"creative_dialogue"`	Cellular_Homeostasis	0.50	Creative/open-ended generation
`"general"`	Best cosine match	varies	General-purpose

`mercy_k` Tuning Guide

`mercy_k`	Contraction Speed	Quality	Diversity	Recommended For
0.25-0.35	Slow	Highest	Lowest	Structured output (code, JSON)
0.40-0.50	Moderate	High	Moderate	Factual QA, mathematical text
0.55-0.65	Fast	Good	Higher	Creative generation, dialogue
> 0.70	Very fast	May overshoot	High	Experimental only

---

6. Integration Protocol with Attention Engine

6.1 Minimal Integration (Synthetic Logits)

```
from pipeline import create_sequential_pipeline

pipeline = create_sequential_pipeline(vocab_size=32768)
result = pipeline.run(context_ids=[42, 17, 305])
next_token = result.final_token
```

6.2 Integration with HuggingFace Transformers

```
from pipeline import ActualizerFDSAQCAPipeline, PipelineConfig, AttentionEngineInterf
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

class HuggingFaceAttentionEngine(AttentionEngineInterface):
    def __init__(self, model_name: str):
        self.model = AutoModelForCausalLM.from_pretrained(model_name)
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        super().__init__(vocab_size=self.model.config.vocab_size)

    def get_logits(self, context_ids, step=0):
        input_ids = torch.tensor([context_ids])
        with torch.no_grad():
            outputs = self.model(input_ids)
        return outputs.logits[0, -1, :].tolist()

    def decode_token(self, token_id):
        return self.tokenizer.decode([token_id])
```

```
attention = HuggingFaceAttentionEngine("meta-llama/Llama-3-8B")
pipeline = ActualizerFDSAQCAPipeline(
    config=PipelineConfig(
        vocab_size=attention.vocab_size,
        execution_mode="parallel",
        K=8,
        context_type="logical_coding",
    ),
    attention_engine=attention,
)

result = pipeline.run(context_ids=[1045, 2079, 1997])
print(f"Next token: '{attention.decode_token(result.final_token)}'")
```

6.3 Providing Pre-computed Logits Directly

```
raw_logits = my_model.forward(context).tolist()  # list[float], len=V
result = pipeline.run(
    context_ids=my_context,
    logits=raw_logits,   # bypass internal attention engine call
)
```

6.4 Grammar Rules Integration

```
grammar_rules = {
    42: {100, 101, 102, 200},   # after token 42, only these tokens are valid
    17: {50, 51, 52},
}
pipeline = ActualizerFDSAQCAPipeline(
    config=PipelineConfig(vocab_size=1000),
    grammar_rules=grammar_rules,
)
```

---

7. Benchmark Protocol

7.1 Running the Benchmark

```
# Quick single run (default settings)
python benchmark_with_attention.py

# Full benchmark suite
python benchmark_with_attention.py --full

# Ordering verification only (Theorem 2.1)
python benchmark_with_attention.py --ordering-only

# Custom parameters
python benchmark_with_attention.py --vocab-size 5000 --K 8 --T 20
```

7.2 Benchmark Dimensions

Dimension 1: Latency (ms/step)

Metric	Description
`mean_baseline_ms`	Greedy argmax from raw softmax (no pipeline)
`mean_seq_ms`	Sequential FDSA + Actualizer pipeline
`mean_par_ms`	Parallel FDSA + QCA + Actualizer pipeline
`speedup_seq_vs_baseline`	Latency improvement: seq pipeline vs greedy baseline
`speedup_par_vs_seq`	Latency improvement: parallel vs sequential

Dimension 2: Quality

Metric	Description	Target
`mean_seq_pruning`	Fraction of vocabulary eliminated by FDSA	> 90%
`seq_actualized_rate`	Fraction of steps reaching actualization branch	> 70%
`mean_seq_valuation`	Mean nu_t at convergence (0=uncollapsed, 1=actualized)	> 0.5

Dimension 3: Scaling

Theoretical: speedup = K (from QCA Theorem 2 Corollary) Measured: 0.5K to 0.8K (accounting for spawn overhead and inter-process communication)

7.3 Important Note on QCA Output

> [!NOTE] The `QCAParallelEngine` operates on a **steering diagnostic** over synthetic per-node logit distributions derived from QCA node coordinates, NOT from a real model forward pass. The QCA layer is a **clustering and parallelization front-end** that distributes Actualizer work across K clusters. Real model logits enter at Stage 1 (FDSA) and Stage 3 (Final Actualizer Snap).

---

8. Engine-by-Engine Reference

8.1 FDSA Pruner (`fdsa_pruner.py`)

Class: `VectorizedFDSAPruner` **Key method:** `prune_numpy(logits, last_token, grammar_rules, context_type)`

Algorithm (V3_U1 §6.7.2):

1. Isomorphic Anchoring: cosine-match context profile -> reference domain -> k_ref
2. Fractal Dimension: D = ln(V) / ln(1/k_ref)
3. Complexity threshold: theta = -D * 1.5
4. Mask: logits[v] = -inf if logits[v] < theta or v not in grammar[last_token]

Performance: O(V) Boolean operations. At V=30,000: **4.56x faster** than raw softmax.

8.2 QCA Parallel Engine (`qca_parallel_engine.py`)

Class: `QCAParallelEngine` **Key method:** `process_parallel(nodes, grammar_rules, verbose)`

Algorithm (CKT White Paper v3 §7.2):

1. Step 1: Distance Matrix — O(N^2) pairwise Euclidean distances
2. Step 2: Quench Binding — T_q^RGG = gamma * sqrt(A * ln(N/K) / (pi * N)); farthest-point seeds; K clusters
3. Parallel Execution: K workers; each runs FDSA + NumpyActualizerEngine on its cluster
4. Synthesis: meta_logits[tok] += valuation + 1.0 across all clusters

Complexity reduction: O(N^2) -> O(N^2/K)

8.3 Actualizer Engine (`numpy_actualizer_engine.py`)

Class: `NumpyActualizerEngine` **Key method:** `steer(logits, history, target_tokens)`

V3_U1 Algorithm Steps:

Step	Operation	Formula
Init	Substrate	U_0 = softmax(logits)
a	Prime projection	alpha = [alpha_O, alpha_J, alpha_M, alpha_K, alpha_P]
b	Structural entropy	H(R) = Var(alpha) + (sum(alpha_i^2) - 1)^2
c	Valuation	nu_t = 1 - H(R)/H_max
d	Drift tensor	D = w_L*D_local* + *w_G*D_global + w_F*D_future
e	Trace	Tr(D_mu_nu) = sum_v U(v)*D(v)
f	Vacuum Brake	U_b(v) = U(v)*exp(-D(v)/tau) / Z
g	Contraction	U_{n+1} = k*U_b + (1-k)*U_n
h	Convergence	delta =
i	Causal Snap	Tr(D) <= tau_bif: S* = argmax U (ACTUALIZED)

Returns: `(token, U_final, Tr_D, iterations, nu_history, is_actualized)`

---

9. Module Structure

```
Final_Output/Actualizer_Engine_FDSA_QCA/
+-- pipeline.py                <- Main pipeline orchestration
|                                 ActualizerFDSAQCAPipeline, PipelineConfig
|                                 AttentionEngineInterface
|                                 create_sequential_pipeline()
|                                 create_parallel_pipeline()
|
+-- benchmark_with_attention.py <- Real benchmark with attention engine
|                                 SimulatedAttentionEngine
|                                 run_attention_benchmark()
|                                 run_scaling_benchmark()
```

```
|                                  verify_canonical_ordering()
|                                  run_full_benchmark()
|
+-- run_pipeline.py             <- CLI quick-start runner
|                                  --mode sequential|parallel|sequence|benchmark|orde
|
+-- USAGE_MANIFESTO.md          <- This document

Dependencies (from ../02_Core_Engine/):
+-- actualizer_engine.py        <- ActualizerEngine (V3_U1 corrected, reference)
+-- numpy_actualizer_engine.py  <- NumpyActualizerEngine (vectorized, 5-10x faster)
+-- fdsa_pruner.py              <- VectorizedFDSAPruner, FractalDeductionSearch
+-- qca.py                      <- QuenchClusterAlgorithm, QCANode, QCACluster
+-- qca_parallel_engine.py      <- QCAParallelEngine (processes + JAX backends)
```

---

10. Quick-Start Examples

Example 1: Minimal Sequential Step

```
from pipeline import create_sequential_pipeline

pipeline = create_sequential_pipeline(vocab_size=1000)
result = pipeline.run(context_ids=[42, 17, 305])
print(result.summary())
# [ACTUALIZED] token=387 | nu=0.7234 | Tr(D)=1.2341 |
#   active_vocab=48 | pruned=95.2% | total=12.34ms
```

Example 2: Parallel Pipeline with Verbose Logging

```
from pipeline import create_parallel_pipeline

pipeline = create_parallel_pipeline(
    vocab_size=2000,
    K=6,
    backend="auto",
    context_type="mathematical",
    verbose=True,
```

```
)
result = pipeline.run(context_ids=[100, 200, 300, 400])
print(f"Final token: {result.final_token}, Actualized: {result.is_actualized}")
```

Example 3: Autoregressive Generation

```
from pipeline import create_sequential_pipeline

pipeline = create_sequential_pipeline(vocab_size=5000, mercy_k=0.45)
prompt = [1, 42, 17, 305]

generated, results = pipeline.generate_sequence(
    prompt_ids=prompt,
    max_new_tokens=20,
    stop_token=2,
)
print(f"Generated IDs: {generated}")
```

Example 4: Raw Logits from External Model

```
from pipeline import ActualizerFDSAQCAPipeline, PipelineConfig

config = PipelineConfig(vocab_size=32768, execution_mode="sequential")
pipeline = ActualizerFDSAQCAPipeline(config=config)

logits = my_transformer.forward(context_ids).tolist()  # list[float], len=32768
result = pipeline.run(context_ids=context_ids, logits=logits)
next_token = result.final_token
```

Example 5: Quick Benchmark

```
from benchmark_with_attention import run_attention_benchmark

bench = run_attention_benchmark(
    vocab_size=1000, T_steps=10, K=5,
    context_type="logical_coding", verbose=True,
)
```

```
print(f"Pruning: {bench.mean_seq_pruning:.1%}")
print(f"Actualization rate: {bench.seq_actualized_rate:.1%}")
```

Example 6: Ordering Verification (Theorem 2.1)

```
from benchmark_with_attention import verify_canonical_ordering

result = verify_canonical_ordering(vocab_size=500, T_steps=5)
print(f"Theorem 2.1 confirmed: {result['theorem_2_1_confirmed']}")
print(f"Reversed overhead: {result['latency_ratio_reversed_over_correct']:.2f}x")
```

---

11. Theoretical Guarantees

Guarantee 1: Banach Convergence (V3_U1 §2.5)

The contractive mapping T_k with k in (0,1) converges to a unique fixed point U* in Delta^(V-1) by the Banach Fixed-Point Theorem:

||U_{n+1} - U*||_2 <= k^n * ||U_0 - U*||_2 -> 0

Guarantee 2: Canonical Ordering (Theorem 2.1)

FDSA-first composition achieves convergence in O(log(1/eps) / log(1/lambda)) iterations. Reversed composition either fails to converge or reaches an incorrect fixed point. Waste bound: N_max * (V-M)/V approx N_max iterations wasted on dead tokens.

Guarantee 3: Bifurcation Criterion (Theorem 3.3)

- Tr(D_mu_nu) <= tau_bif: actualization branch -> S* = argmax U (unique selection)
- Tr(D_mu_nu) > tau_bif: dissolution branch -> fallback token (branch rejected)

Guarantee 4: QCA Complexity Reduction (CKT §7.2 Theorem 2)

K parallel clusters each solve size N/K at cost O((N/K)^2); aggregate = O(N^2/K) — factor-K improvement over sequential O(N^2).

Guarantee 5: Structural Entropy Minimum (V3_U1 §3.3)

H(R) = Var(alpha) + (sum(alpha_i^2) - 1)^2 = 0 if and only if alpha_i = 1/sqrt(5) for all i (symmetric equilibrium, Theorem 3.2). The Actualizer steers toward this minimum-entropy fixed point.

---

12. Known Limitations and Edge Cases

12.1 Total Vocabulary Collapse

Trigger: FDSA grammar rules too restrictive, or anchor_threshold too aggressive. **Symptom:** `active_count = 0` from `prune_numpy()`. **Resolution:** `VectorizedFDSAPruner` falls back to pure logit-threshold gate if no tokens survive.

12.2 QCA Cluster Imbalance

Trigger: N < K (fewer active tokens than clusters requested). **Symptom:** `QCA requires N >= K` ValueError. **Resolution:** `_build_qca_nodes()` always generates `max(K+1, K*3)` nodes; guaranteed N >= K.

12.3 Parallel Mode on Windows

Issue: Python multiprocessing fork context can corrupt JAX state. **Resolution:** `qca_parallel_engine.py` uses `multiprocessing.get_context("spawn")`. Spawn overhead ~100-200ms per call.

12.4 Non-Convergence Detection

Trigger: Pathological logit distribution or numerical precision. **Resolution:** `max_iters` ceiling guarantees termination; at exhaustion, snaps to argmax(U) with `is_actualized = (Tr_D <= tau_bif)` flag.

12.5 Attractor Over-Concentration

Trigger: Context produces near-degenerate attractor distribution. **Resolution:** `NumpyActualizerEngine` applies smoothing S_c = (1-alpha)*S_c* + *alpha*uniform when max(S_c) > 0.9, with alpha = 0.1.

Summary Reference Table

Component	Module	Class / Function	Role in Pipelin
Pipeline Orchestrator	`pipeline.py`	`ActualizerFDSAQCAPipeline`	Coordinates all three stages
Configuration	`pipeline.py`	`PipelineConfig`	All pipeline hyperparamete
Attention Interface	`pipeline.py`	`AttentionEngineInterface`	Logit source (override for rea models)
FDSA Pruner	`fdsa_pruner.py`	`VectorizedFDSAPruner`	Stage 1: vocabulary reduction
QCA Crystallizer	`qca.py`	`QuenchClusterAlgorithm`	Stage 2: problem partitioning
Parallel Engine	`qca_parallel_engine.py`	`QCAParallelEngine`	Stage 2: paralle per-cluster steering
Actualizer (NumPy)	`numpy_actualizer_engine.py`	`NumpyActualizerEngine`	Stage 3: final contractive snaj
Actualizer (Python)	`actualizer_engine.py`	`ActualizerEngine`	Reference implementatior
Benchmark	`benchmark_with_attention.py`	`run_full_benchmark()`	Full benchmark suite
Runner	`run_pipeline.py`	CLI	Quick-start and demos

End of Usage Manifesto — Actualizer_Engine_FDSA_QCA v1.0.0 Consciousness and Prime Base Intelligence Research Framework — July 2026 Author: Mohamed Gamal Eldin Abdelaziz Noureldin | ORCID: 0009-0006-3991-1153